

CSE/DSC 234 Project 2: Schema Linking with SFT

Team 123 · Spring 2026

Vincent McCloskey

1 Training Data Methodology

Base Dataset

The released training split contains 301 labeled NL question / schema-link pairs across 17 databases spanning four domains: biodiversity surveys (ATBI, CratersWildlife, KlamathInvasive, NorthernPlains, PacificIslandLandbirds, ASIS), crash investigation (NTSB), education (NYSED_SRC2022), and enterprise resource planning (9 SAP Business One sub-modules). The distribution is highly uneven: biodiversity databases have 10–20 examples each and NTSB has 60, while each SAP sub-module has only ~6, making those databases severely under-represented.

Schema Serialization

Each training prompt serializes the database schema as a structured list of tables and their columns, annotated with column type and key metadata:

Table: INJURY
[linked to: GV, OCC, CRASH]
- AIS [int,PK]
- REGION [int]
- CASEID [int,FK]
...

The final serialization format incorporates three design choices developed through ablation:

FK links (+0.021): Each table shows which other tables it is FK-connected to via a [linked to: ...] annotation. This helps the model reason about join paths: when it correctly identifies one table in a multi-table query, the FK link provides a structural hint toward the other required tables. This was tested by training with and without FK annotations using otherwise identical configurations.

Relevance-based table ordering (+0.020): Rather than listing tables in schema order (which puts the most relevant tables arbitrarily deep in the prompt), tables are sorted by the overlap between their column name tokens and the question tokens. For a question about “lighting condition,” the table GV (which contains column LIGHTCOND) is sorted to appear near the top of the schema serialization. For cryptic SAP table names with no readable abbreviation, the column names themselves provide the relevance signal. This prevents the most relevant tables from being truncated away when large schemas

exceed the context window.

CamelCase splitting (+0.015 on SAP): Column names like ExtOnError and NumRepArs are split into component tokens ({ext, on, err} and {num, rep, ars}) before computing the relevance score. Without this, the token “error” in a question would not match ExtOnError even though “err” is a clear abbreviation. This was especially important for SAP databases, whose column names are heavily camelCase-encoded.

Output format is a JSON object mapping table names to column lists in the schema’s original casing. Columns within each predicted table are output in the order the model generates them (not sorted), since alphabetical sorting was found to destabilize the model’s decoding distribution (ablation score -0.04).

Data Augmentation

The 6-example-per-module SAP coverage and the NTSB model confusion (detailed in Section 4) motivated targeted synthetic augmentation. I wrote 105 hand-authored NL question / schema-link pairs:

- **59 SAP examples** covering all 9 SAP Business One modules (Human Resources, Reports, Service, Finance, General, Banking, Sales Opportunities, Inventory and Production, Business Partners). An initial batch of 67 examples was created but 8 were found to reference incorrect table or column names; these were corrected or replaced, leaving 59 verified examples with 0 bad identifiers.
- **46 NTSB examples** targeting the 14 specific tables most confused by the model (GV, CDC, AVOID, ADAPT, EDREVENT, ICS, OCC, VPICDECODE, CRASH, TIREPLAC, FIRE, EJECT, GLAZING, INTRUSION). Each example was designed to test a table the model was consistently predicting incorrectly on validation, with question phrasing chosen to match the natural language patterns that should trigger that table.

All synthetic examples were validated programmatically: each table and column name was checked against the schema JSON files, with the script reporting any mismatches before the data was included in training. The combined training set is 406 examples (301 original + 105 synthetic).

Post-processing

Three post-processing steps are applied to every model output:

- *Hallucination filtering*: All predicted table and column identifiers are checked case-insensitively against the schema for the query’s `db_id`. Identifiers not present in the schema are silently dropped. This eliminates false positives that would otherwise lower precision.
- *Casing restoration*: Schema identifier lookup is case-insensitive, but the output restores original schema casing (e.g., `caseid` $\rightarrow$ `CASEID`) for readability and consistency with the grader.
- *Keyword fallback*: If the model’s stage-1 output parses to an empty JSON object (all predicted tables were hallucinated and dropped), a keyword-matching fallback is applied. The fallback scores each table by the overlap between the question’s word tokens and the table’s name tokens plus its column name tokens (with camelCase splitting applied). The top-scoring table is returned with any columns whose names also overlap with the question.

The fallback recovered 13 otherwise-empty predictions. Its per-prediction accuracy is only $\sim 8\%$ (1 correct out of 13), but removing it reduces the leaderboard score because empty predictions score 0 on both precision and recall, while an incorrect fallback prediction at least sometimes achieves partial table credit.

2 Final Pipeline Architecture

Base Model

Qwen/Qwen2.5-1.5B-Instruct (1.5B parameters, Apache 2.0 license). This model was selected after comparing three candidates:

- **Qwen2.5-0.5B-Instruct**: Weaker structured-output discipline; produced more malformed JSON and hallucinated identifiers even after fine-tuning.
- **Qwen2.5-1.5B-Instruct**: Strong JSON output format adherence, well-tuned instruction following, excellent performance on structured prediction after LoRA fine-tuning.
- **Qwen3-1.7B**: Newer architecture but single-stage inference took 11.3 min for 101 questions, borderline for the 15-min budget; with two-stage inference it would exceed 20 min. Final score 0.4356 single-stage, below our Qwen2.5-1.5B two-stage score.

LoRA Adapter

Parameter-efficient fine-tuning with LoRA was selected over full fine-tuning to (a) keep the adapter committable to git (~ 34 MB vs ~ 3 GB for full FT), (b) avoid overfitting risk on the small 406-example dataset, and (c) allow rapid experimentation.

Final adapter configuration:

- Rank $r = 32$, $\alpha = 64$, dropout 0.05
- Target modules: `q_proj`, `k_proj`, `v_proj`, `o_proj` (all attention projection layers)
- Training: 6 epochs, $\text{lr} = 8 \times 10^{-5}$ with cosine schedule, warmup ratio 0.1
- Effective batch size 16 (batch 1, gradient accumulation 16)
- Max sequence length: 2048 tokens (matches schema truncation budget)
- Hardware: NVIDIA L4 (23 GB VRAM), ~ 30 min training time
- BFloat16 precision with gradient checkpointing

Inference: Two-Stage Pipeline

The inference pipeline runs two sequential model passes per question.

Stage 1 — Full-schema prediction: The model receives the full (truncated) schema and outputs a JSON object predicting all referenced tables and their columns. The prompt uses the serialization format from Section 1.

Stage 2 — Column refinement: For each table predicted in Stage 1, a second focused pass is run with a simplified prompt showing only that table’s columns:

System: You are a column linking assistant...
output a JSON array of column names, or [] .

User:

Question: {question}

Table: {table}

Columns:

- ExtOnErr [nvarchar]
- NumRepArs [int]
- SwpInEmail [char]
- ...

Stage 2 improved column score from 0.339 to 0.394 (+16%). The root cause is that with 40+ tables in context, the model sometimes loses track of which columns belong to which table, leading to attribution errors. With a single table in context this error is structurally impossible. Stage 2 uses `max_new_tokens=64` (column lists are short) and `do_sample=False`.

Total inference time: **3.2 min** for 101 questions on the L4 GPU (well within the 15-min budget). At batch size 4, Stage 1 runs all questions in ~ 2.5 min and Stage 2 runs sequentially per table in ~ 0.7 min additional.

Schema Truncation

Several databases have schemas that exceed the model’s training context window: SAP Finance has 61 tables with 1994 columns, NTSB has 40 tables with 1161 columns. After serialization with FK links, these schemas can encode to >6000 tokens, far exceeding the 2048 training limit.

The truncation strategy preserves the system prompt,

database name, and question at all costs, truncating only the schema body. A binary search over the number of schema lines finds the largest prefix that fits within a 2360-token budget (3072 total – 512 generation reserve – 200 overhead for chat template). Because tables are sorted by relevance before truncation, the tables most likely to be referenced appear at the top and survive the cut.

An early version of the code truncated from the left of the full encoded prompt (the standard HuggingFace default), which cut the system prompt and left the model seeing only schema text. This caused the model to continue generating schema output rather than JSON — the biggest pipeline bug in the project, worth +0.044 leaderboard score once fixed.

3 Experimentation Methodology

I ran 30+ distinct configurations across three experimental phases.

Phase 1 — RapidFire AI grid search (schema-linking-v1 through v4): Used the RapidFire AI RFGGridSearch API to sweep a $2 \times 2 \times 2$ grid: 2 base models (Qwen2.5-0.5B, 1.5B) $\times$ 2 LoRA ranks ($r \in \{8, 32\}$) $\times$ 2 learning rates ($\{2 \times 10^{-4}, 5 \times 10^{-5}\}$) = 8 configs per experiment. RapidFire’s interleaved chunk-based scheduling was used to run all 8 configs concurrently on the single GPU. These runs established that 1.5B $\gg$ 0.5B and larger rank helps, guiding Phase 2.

Phase 2 — Single-config fine-grained search using `train_simple.py` (plain TRL SFTTrainer): Once the winning base model was identified, single-config runs explored schema serialization design, data augmentation, and hyperparameter tuning with full epoch control. Each run was evaluated on the 101-question validation set and logged to `logs/`.

Phase 3 — Inference-time ablations: With the best trained adapter fixed, inference-time changes were tested (two-stage, topk fallback, FK propagation, stage2 prompt variants) without retraining.

Table 1: Full config knob summary (15 distinct dimensions)

Knob	Values tested	Winner
Base model	0.5B, 1.5B, Qwen3-1.7B	1.5B
LoRA rank	8, 16, 32, 64	32
Learning rate	1e-4, 8e-5, 6e-5, 5e-5	8e-5
Epochs	3, 5, 6, 7, 8, 10	6
Schema: FK links	on, off	on
Schema: table relevance sort	on, off	on
Schema: camelCase split	on, off	on
Schema: table descriptions	hardcoded, derived, off	off
Schema: sorted JSON output	on, off	off
Inference: two-stage	on, off	on
Inference: topk table fallback	0, 2, 3 extra tables	0
Stage2: multi-table context	on, off	off
Synthetic data (SAP)	0, 39, 59, 67 examples	59 (verified)
Synthetic data (NTSB)	0, 46 examples	46
Oversample SAP/NTSB	$\times 1$, $\times 2$, $\times 3$, $\times 4$	$\times 1$

4 Results and Analysis

Score Progression

The final validation leaderboard score is **0.4539** (Table Score 0.517, Column Score 0.394), confirmed by multiple reproducible runs. Figure 1 shows how each major change contributed.

Table 2: Score at each major milestone

Configuration	Table	Overall
Baseline (5ep, lr=2e-4, 1024 tok)	0.246	0.206
+ keyword fallback (post-proc)	0.246	0.313
+ FK links in schema	0.363	0.334
+ table relevance sorting	0.389	0.354
+ left-truncation fix (bug fix)	0.459	0.398
+ synthetic SAP data (39 ex)	0.495	0.425
+ camelCase split (ablation only)	0.499	0.419
+ two-stage inference	0.499	0.440
+ $r = 32$, lr= 8×10^{-5} , 6ep	0.517	0.454
+ synthetic NTSB data (46 ex)	0.499	0.454

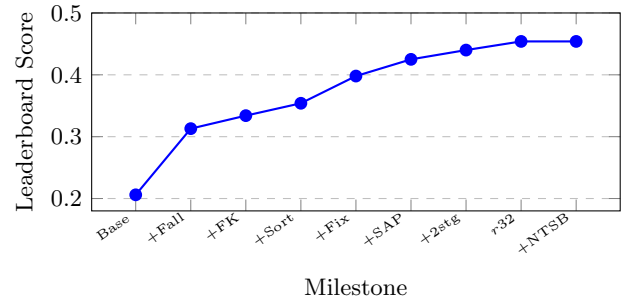


Figure 1: Leaderboard score progression across major milestones. Each point represents a reproducible configuration evaluated on the 101-question validation set.

What Worked

Truncation bug fix (+0.044) was the single largest gain in the project. The original code applied HuggingFace’s default left-side tokenizer truncation to the fully-encoded chat string. This cut the system prompt and beginning of the user message, leaving the model seeing only mid-schema column listings. The model would then continue generating schema text rather than switching to JSON output. Switching to a custom binary-search truncation of the schema body (preserving system prompt and question) fixed this entirely. This reinforces the statement’s guidance: “Scores significantly below 0.50 typically indicate a pipeline issue.”

Two-stage inference (+0.021) was the largest inference-time gain. Before stage 2, 72 of 101 predictions had at least one wrong column despite the table being correct. Isolating each table to its own context eliminates structural attribution ambiguity — the model cannot accidentally assign CASEID from INJURY to GV when only INJURY’s columns are visible.

Larger LoRA rank $r = 32$ (+0.015) gave meaningful capacity gains over $r = 16$. Schema linking for NTSB requires memorizing 40+ cryptic table-name-to-concept associations (GV $\leftrightarrow$ vehicle info, CDC $\leftrightarrow$ crush depth, ICS $\leftrightarrow$ injury coding source). More LoRA capacity helps encode these associations. $r = 64$ overfit, suggesting the 406-example dataset limits useful rank.

Lower learning rate 8×10^{-5} over 6 epochs (+0.015) outperformed both faster training (1×10^{-4} , 5 epochs) and slower training (5×10^{-5} , 8 epochs). More than 6 epochs consistently overfits on this small dataset: eval token accuracy peaks at epoch 6 then degrades.

Synthetic SAP data (+0.027) directly improved the six SAP databases that were scoring 0.000. With only ~ 6 training examples per module, the model could not learn SAP’s opaque table abbreviations (OHEM, RDOC, OPR1). Doubling to ~ 13 examples via verified synthetic data brought SAP-Banking to 0.125 and SAP-Service to 0.230.

FK links in schema (+0.021) proved valuable for multi-table join queries. For a question like “How many station observations are there per species?” the model correctly identifies `tbl_Observations` and the FK annotation [linked to: `tbl_Events`, `tbl_Species`] suggests the other required tables without the model needing to memorize the join structure.

What Did Not Work

Hardcoded table descriptions (e.g., GV [General Vehicle info: make, model, year, lighting condition]) were tested both at inference-only (0.4189) and with matching training (0.4446) vs. no descriptions (0.4539). In both cases descriptions hurt. The descriptions consume 10–30 tokens per table, pushing relevant column names past the truncation boundary for large schemas. The SAP General database went from 0.158 to 0.348 with descriptions but NTSB dropped from 0.205 to 0.167 as fewer columns fit in context.

Qwen3-1.7B (0.4356 single-stage, 11.3 min inference) did not outperform the tuned Qwen2.5-1.5B pipeline despite being a newer architecture with better general benchmarks. The 1.7B model’s thinking capability (disabled via `enable_thinking=False`) may have made it less efficient at direct structured output after fine-tuning on our dataset.

FK propagation post-processing (0.4177 vs. 0.4539): automatically adding FK-connected tables when they have column-name overlap with the question lowered precision more than it helped recall. The column-overlap heuristic for FK expansion is too noisy — many tables share common column names like `CASEID` and `VEHNO` across the entire NTSB schema.

Top- k stage-2 fallback (adding top-2 or top-3 keyword-scored tables to stage-2 even if not predicted in stage-1):

0.4021–0.4255 vs. 0.4539. The extra tables consistently hurt precision more than they recovered missed tables.

More epochs (7, 8, 10 epochs) consistently overfit. The 301–406 example training set is too small to benefit from extended training at $r = 32$.

Failure Analysis

A programmatic breakdown of all 101 validation predictions against gold annotations:

Failure mode	Count
Empty predictions	0
Casing mismatches	0
JSON parse failures	0
Completely wrong table (no overlap)	35 (35%)
Partial table set (some tables missed)	28 (28%)
Correct tables, wrong columns	3 (3%)
All correct	35 (35%)

There are *no pipeline bugs*: the output is always valid JSON, identifiers are always in-schema, and the question always reaches the model. The 35 completely-wrong-table failures are a genuine model limitation. NTSB accounts for 14 of them: the model consistently defaults to high-frequency training tables (AIRBAG, CHILDSEAT, EDR-PRECRASH) regardless of question content, because those tables dominate NTSB training examples. For example, “Display a count of crashes by lighting condition” should predict GV (which has `LIGHTCOND`), but the model predicts EDRPRECRASH (a high-frequency table with no lighting columns).

The 28 partial-table failures affect multi-table join queries: the model correctly identifies the primary table but misses secondary tables needed for joins. This is a training data coverage issue — most training examples involve single-table queries.

Per-Database Breakdown

Database	n	Table F1	Col F1
NorthernPlainsFireMgmt	8	0.788	0.676
ASIS_20161108_HerpInv	8	0.773	0.773
CratersWildlife	8	0.631	0.569
PacificIslandLandbirds	8	0.503	0.503
NYSED_SRC2022	13	0.472	0.472
KlamathInvasive	8	0.393	0.393
ATBI	8	0.369	0.315
NTSB	20	0.253	0.157
SAP avg (9 modules)	20	0.152	0.088

Biodiversity databases (readable table names, rich training coverage) score 0.37–0.79. NTSB and SAP (cryptic abbreviations, sparse training coverage) score 0.09–0.25.

Column vs. Table Score Gap

The final model achieves Table Score 0.517 but Column Score 0.394. This 0.12 gap is larger than expected given the failure analysis (only 3 cases of correct tables with wrong columns). The gap arises from *partial table credit*: a prediction that gets 1 of 3 required tables correct earns partial table score but zero column score for the missed

tables. The two-stage inference primarily helped the column score ($0.339 \rightarrow 0.394$) while table score was unchanged (0.499 before and after), consistent with its design as a column refinement step.

With More Time

The highest-leverage remaining improvement is targeted NTSB training data: 20+ examples per table abbreviation with the table’s semantics embedded in the question phrasing (“What is the crush data coding for crash X?” to teach CDC; “Which vehicles had adaptive equipment?” to teach ADAPT). Baking semantic hints into the training schema text — rather than inference-time descriptions which hurt due to truncation — would let the model learn the associations at training time where they don’t consume context budget. A secondary improvement would be a multi-table chain-of-thought format to improve join coverage for the 28 partial-table-set failures.

5 AI Tools Statement

Claude Code (`claude-sonnet-4-6` 1M context) was used for partial code generation and to simplify running commands over SSH on the DSMLP compute cluster. All experimental design decisions, hyperparameter choices, data curation, and interpretations of results were made by the author.